MODULE *IntegrationContract*

Section: Imports

Shared operators used by this module are imported below.

EXTENDS *Naturals*, *FiniteSets*, *FailurePolicy*

Section: Integration boundary

This layer receives completed per-check snapshots from *Overwatch*. It specifies requests, responses, delivery channels, and session lifecycle for every host. Both layers use the same *FailurePolicy* definitions.

CONSTANTS *Hosts*, *Checks*, *Names*, *MaxRequests*, *MaxChanges*, *MaxSessions*

$$AllHosts \triangleq \{\text{"claude"}, \text{"codex"}, \text{"opencode1"}, \text{"opencode2"}, \text{"pi"}, \text{"neovim"}\}$$

ASSUME $Hosts \subseteq AllHosts \land Hosts \neq \{\}$

These are host capabilities, not advisory/strict policy modes. A warning cannot be counted as a denied completion, and an editor has no agent gate.

Section: Host capabilities

The host determines whether protected requests wait for a running check, how guarded completion is handled, and which delivery channels exist. A host notices another process's change when it can tell it from the session's own: *Claude* Code reads each watcher result against the session's edits, *Codex* sees a change first at a hook that follows no edit of the session's, and an editor treats every save alike. Other hosts read every result as the session's. These are integration capabilities, not user-selectable enforcement modes.

$$\begin{aligned}
&Capabilities(h) \triangleq \\
&\quad [agent \mapsto h \neq \text{"neovim"}, \\
&\quad\ waitsForCheck \mapsto h \in \{\text{"codex"}, \text{"opencode1"}, \text{"opencode2"}, \text{"pi"}\}, \\
&\quad\ completion \mapsto \text{CASE } h \in \{\text{"claude"}, \text{"codex"}\} \to \text{"block"} \\
&\qquad\qquad\qquad\ \square\ h = \text{"opencode1"} \to \text{"warn"} \\
&\qquad\qquad\qquad\ \square\ h = \text{"opencode2"} \to \text{"queue"} \\
&\qquad\qquad\qquad\ \square\ h = \text{"pi"} \to \text{"continue"} \\
&\qquad\qquad\qquad\ \square\ \text{OTHER} \to \text{"none"}, \\
&\quad\ delivery \mapsto \text{CASE } h = \text{"claude"} \to \text{"turn"} \\
&\qquad\qquad\qquad\ \square\ h = \text{"opencode2"} \to \text{"context"} \\
&\qquad\qquad\qquad\ \square\ h = \text{"neovim"} \to \text{"quickfix"} \\
&\qquad\qquad\qquad\ \square\ \text{OTHER} \to \text{"event"}, \\
&\quad\ idleWake \mapsto h \in \{\text{"claude"}, \text{"opencode2"}\}, \\
&\quad\ status \mapsto h \in \{\text{"claude"}, \text{"pi"}, \text{"neovim"}\}, \\
&\quad\ noticesOutside \mapsto h \in \{\text{"claude"}, \text{"codex"}, \text{"neovim"}\}]
\end{aligned}$$

$$\begin{aligned}
&Kinds \triangleq \{\text{"inspect"}, \text{"repair"}, \text{"publish"}, \text{"complete"}\} \\
&Empty \triangleq [c \in Checks \mapsto \{\}] \\
&Snapshots \triangleq [Checks \to \text{SUBSET } Names] \\
&NoReply \triangleq [ready \mapsto \text{FALSE}, decision \mapsto \text{"none"}]
\end{aligned}$$

Section: State shape

Request and reply are separate so that a check can finish or a session can close between them. Report visibility, deferred context, corrections, and idle wakeups are separate observations.

VARIABLES *host*, *active*, *busy*, *reports*, *baseline*, *inherited*, *selected*,
request, *reply*, *requests*, *changes*, *session*,
visible, *contextPending*, *contextSent*, *correctionQueued*,
contexts, *warnings*, *corrections*, *inTurn*, *wakes*, *work*,
correctionPending, *correctionTurn*, *outside*, *outsides*

$$\begin{aligned}
vars \triangleq \langle &host, active, busy, reports, baseline, inherited, selected, \\
&request, reply, requests, changes, session, visible, \\
&contextPending, contextSent, correctionQueued, contexts, \\
&warnings, corrections, inTurn, wakes, work, \\
&correctionPending, correctionTurn, outside, outsides\rangle
\end{aligned}$$

Section: *Guard* selection and inherited policy

An empty selection means all checks. *FailureGuards* decides whether each selected check guards publication; *Notify* identifies new failures for delivery.

$$Guarded(scope) \triangleq$$
$$\{c \in scope : FailureGuards(reports[c],\ baseline[c],\ inherited)\}$$
$$Guard \triangleq Capabilities(host).agent \land Guarded(Checks) \neq \{\}$$
$$Notify \triangleq Capabilities(host).agent \land$$
$$(\exists\, c \in Checks : FailureGuards(reports[c],\ baseline[c],\ \text{“allow”}))$$
$$SelectedChecks \triangleq \text{IF } selected = \{\} \text{ THEN } Checks \text{ ELSE } selected$$

Section: State domains

Counters and snapshots are bounded for *TLC* exploration. These bounds do not assert a runtime limit on requests or sessions.

$$TypeOK \triangleq$$
$$\land\ host \in Hosts \land active \in \text{BOOLEAN} \land busy \in \text{BOOLEAN}$$
$$\land\ work \in 0\,..\,((MaxSessions + 1) * MaxRequests)$$
$$\land\ inTurn \in \text{BOOLEAN}$$
$$\land\ wakes \in 0\,..\,(MaxChanges + MaxSessions + 1 + (MaxSessions + 1) * MaxRequests)$$
$$\land\ reports \in Snapshots \land baseline \in Snapshots$$
$$\land\ inherited \in \{\text{“allow”},\ \text{“block”}\} \land selected \subseteq Checks$$
$$\land\ request \in Kinds \cup \{\text{“none”}\}$$
$$\land\ reply \in [ready : \text{BOOLEAN},$$
$$decision : \{\text{“none”},\ \text{“allow”},\ \text{“deny”},\ \text{“block”},\ \text{“warn”},\ \text{“queue”},\ \text{“continue”},\ \text{“cancelled”}\}]$$
$$\land\ requests \in 0\,..\,MaxRequests \land changes \in 0\,..\,MaxChanges \land session \in 0\,..\,MaxSessions$$
$$\land\ visible \in [known : \text{BOOLEAN},\ failures : Snapshots]$$
$$\land\ contextPending \in \text{BOOLEAN} \land contextSent \in \text{BOOLEAN} \land correctionQueued \in \text{BOOLEAN}$$
$$\land\ contexts \in 0\,..\,(MaxChanges + MaxSessions + 1)$$
$$\land\ warnings \in 0\,..\,((MaxSessions + 1) * MaxRequests)$$
$$\land\ corrections \in 0\,..\,((MaxSessions + 1) * MaxRequests)$$
$$\land\ correctionPending \in \text{BOOLEAN} \land correctionTurn \in \text{BOOLEAN} \land outside \in \text{BOOLEAN}$$
$$\land\ outsides \in 0\,..\,MaxChanges$$

Section: Initial state

A host starts active with arbitrary reports, baseline, policy, and check state. No reply, visible report, deferred context, or correction has been produced.

$$Init \triangleq$$
$$\land\ host \in Hosts \land active = \text{TRUE} \land busy \in \text{BOOLEAN}$$
$$\land\ reports \in Snapshots \land baseline \in Snapshots$$
$$\land\ inherited \in \{\text{“allow”},\ \text{“block”}\} \land selected \in \text{SUBSET } Checks$$
$$\land\ request = \text{“none”} \land reply = NoReply$$
$$\land\ requests = 0 \land changes = 0 \land session = 0$$
$$\land\ visible = [known \mapsto \text{FALSE},\ failures \mapsto Empty]$$
$$\land\ contextPending = \text{FALSE} \land contextSent = \text{FALSE} \land correctionQueued = \text{FALSE}$$
$$\land\ contexts = 0 \land warnings = 0 \land corrections = 0 \land wakes = 0 \land work = 0 \land inTurn \in \text{BOOLEAN}$$
$$\land\ correctionPending = \text{FALSE} \land correctionTurn = \text{FALSE} \land outside = \text{FALSE} \land outsides = 0$$

Section: Requests and completion decisions

Inspection and repair remain answerable during checks. Protected requests follow the host waiting capability and consult the selected reports. Respond records the decision and accounts separately for warnings, corrections, and wakeups; *Acknowledge* releases the request slot.

$$Request(k) \triangleq$$
$$\land\ active \land request = \text{“none”} \land requests < MaxRequests$$
$$\land\ request' = k \land reply' = NoReply \land requests' = requests + 1$$
$$\land\ \text{UNCHANGED } \langle host,\ active,\ busy,\ reports,\ baseline,\ inherited,\ selected,$$
$$changes,\ session,\ visible,\ contextPending,\ contextSent,$$
$$correctionQueued,\ contexts,\ warnings,\ corrections,\ inTurn,\ wakes,\ work,$$
$$correctionPending,\ correctionTurn,\ outside,\ outsides\rangle$$

```
CanRespond ≜
    active ∧ request ≠ "none" ∧ ¬reply.ready ∧
    (request ∈ {"inspect", "repair"}      ∨ ¬Capabilities(host).agent ∨
     ¬Capabilities(host).waitsForCheck ∨ ¬busy)

Decision ≜
    CASE request ∈ {"inspect", "repair"} → "allow"
      □  request = "publish" →
           IF Capabilities(host).agent ∧ Guarded(SelectedChecks) ≠ {}
            THEN "deny" ELSE "allow"
      □  request = "complete" → IF Guard THEN Capabilities(host).completion ELSE "allow"

QueuesCorrection ≜ Decision = "queue" ∨ (host = "claude" ∧ Decision = "block")
EndsTurn ≜ request = "complete" ∧ (Decision ∈ {"allow", "warn", "queue"} ∨ host = "claude")

Respond ≜
    ∧ CanRespond
    ∧ inTurn' = IF EndsTurn THEN FALSE ELSE inTurn
    ∧ correctionTurn' = IF EndsTurn THEN FALSE ELSE correctionTurn
    ∧ correctionPending' = (correctionPending ∨ (QueuesCorrection ∧ ¬correctionQueued))
    ∧ reply' = [ready ↦ TRUE, decision ↦ Decision]
    ∧ warnings' = warnings + IF Decision = "warn" THEN 1 ELSE 0
    ∧ corrections' = corrections + IF QueuesCorrection ∧ ¬correctionQueued THEN 1 ELSE 0
    ∧ correctionQueued' = (correctionQueued ∨ QueuesCorrection)
    ∧ wakes' = wakes + IF QueuesCorrection ∧ ¬correctionQueued ∧
                          (host = "opencode2" ∨ ¬inTurn)
                       THEN 1 ELSE 0
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 request, requests, changes, session, visible,
                 contextPending, contextSent, contexts, work, outside, outsides⟩

Acknowledge ≜
    ∧ request ≠ "none" ∧ reply.ready
    ∧ request' = "none"
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 reply, requests, changes, session, visible,
                 contextPending, contextSent, correctionQueued,
                 contexts, warnings, corrections, inTurn, wakes, work,
                 correctionPending, correctionTurn, outside, outsides⟩
```

Section: *Checks* and result delivery

A completed check replaces the reports. Changed findings invalidate old delivery and correction deduplication. Report updates visibility and either publishes event context or retains it for *Deliver*. Recovery clears stale context.

```
BeginCheck ≜
    ∧ active ∧ ¬busy ∧ changes < MaxChanges
    ∧ busy' = TRUE
    ∧ UNCHANGED ⟨host, active, reports, baseline, inherited, selected,
                 request, reply, requests, changes, session, visible,
                 contextPending, contextSent, correctionQueued,
                 contexts, warnings, corrections, inTurn, wakes, work,
                 correctionPending, correctionTurn, outside, outsides⟩
```

The result-ordering and baseline rules are specified in *Overwatch.tla*. This layer accepts a completed result at that boundary, then constrains what every host must expose and which requests it may hold behind a check.

```
FinishCheck(f) ≜
    ∧ active ∧ busy ∧ changes < MaxChanges
    ∧ reports' = f ∧ busy' = FALSE ∧ changes' = changes + 1 ∧ outside' = FALSE
    ∧ IF f ≠ reports
```

```
            THEN  ∧ contextSent′ = FALSE ∧ correctionQueued′ = FALSE
                  ∧ contextPending′ = FALSE
            ELSE  UNCHANGED ⟨contextSent, correctionQueued, contextPending⟩
      ∧ UNCHANGED ⟨host, active, baseline, inherited, selected, request,
                   reply, requests, session, visible, contexts, warnings, corrections, inTurn, wakes, work,
                   correctionPending, correctionTurn, outsides⟩
```

A check that finishes on another process's change, with no session edit since the last result, replaces the reports for status and guards alone: it neither releases correction deduplication nor counts as a change the session made, and *Report* gives it no failure context. A context still waiting for the old reports is dropped, as a newer result replaces it.

```
OutsideFinish(f) ≜
    ∧ active ∧ busy ∧ outsides < MaxChanges ∧ Capabilities(host).noticesOutside
    ∧ reports′ = f ∧ busy′ = FALSE ∧ outside′ = TRUE ∧ outsides′ = outsides + 1
    ∧ contextPending′ = (contextPending ∧ f = reports)
    ∧ UNCHANGED ⟨host, active, baseline, inherited, selected, request,
                 reply, requests, changes, session, visible, contextSent,
                 correctionQueued, contexts, warnings, corrections, inTurn, wakes, work,
                 correctionPending, correctionTurn⟩

Told ≜ Notify ∧ ¬outside
Report ≜
    ∧ active ∧ ¬busy
    ∧ visible′ = [known ↦ TRUE, failures ↦ reports]
    ∧ contextPending′ = (Told ∧ ¬contextSent ∧ Capabilities(host).delivery ∈ {"turn", "context"})
    ∧ contextSent′ = (contextSent ∨ (Told ∧ Capabilities(host).delivery = "event"))
    ∧ contexts′ = contexts + IF Told ∧ ¬contextSent ∧ Capabilities(host).delivery = "event" THEN 1 ELSE 0
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 request, reply, requests, changes, session,
                 correctionQueued, warnings, corrections, inTurn, wakes, work,
                 correctionPending, correctionTurn, outside, outsides⟩

Deliver ≜
    ∧ active ∧ contextPending
    ∧ wakes′ = wakes + IF host = "claude" ∧ ¬inTurn THEN 1 ELSE 0
    ∧ contextSent′ = TRUE ∧ contextPending′ = FALSE ∧ contexts′ = contexts + 1
    ∧ UNCHANGED ⟨host, active, busy, reports, baseline, inherited, selected,
                 request, reply, requests, changes, session, visible,
                 correctionQueued, warnings, corrections, inTurn, work,
                 correctionPending, correctionTurn, outside, outsides⟩
```

Section: Shutdown restart and new work

Stop cancels an outstanding request and clears visible and pending delivery. Restart preserves project policy, takes reports as the new baseline, and resets delivery state. A submitted correction runs in a turn of its own, which on *OpenCode* 2 is always the next; any other turn is the person's. Completed tool work permits a fresh correction, except in the correction's own turn, where inspecting and repairing are its answer.

```
Stop ≜
    ∧ active
    ∧ inTurn′ = FALSE
    ∧ correctionPending′ = FALSE ∧ correctionTurn′ = FALSE
    ∧ active′ = FALSE ∧ busy′ = FALSE ∧ request′ = "none"
    ∧ reply′ = [ready ↦ TRUE, decision ↦ "cancelled"]
    ∧ visible′ = [known ↦ FALSE, failures ↦ Empty]
    ∧ contextPending′ = FALSE
    ∧ UNCHANGED ⟨host, reports, baseline, inherited, selected, requests,
                 changes, session, contextSent, correctionQueued,
                 contexts, warnings, corrections, wakes, work, outside, outsides⟩

Restart ≜
```

$\land \neg active \land session < MaxSessions$
$\land inTurn' = \text{FALSE}$
$\land correctionPending' = \text{FALSE} \land correctionTurn' = \text{FALSE}$
$\land active' = \text{TRUE} \land session' = session + 1 \land busy' = \text{FALSE}$
$\land baseline' = reports$
$\land request' = \text{“none”} \land reply' = NoReply \land requests' = 0$
$\land visible' = [known \mapsto \text{FALSE}, failures \mapsto Empty]$
$\land contextPending' = \text{FALSE} \land contextSent' = \text{FALSE} \land correctionQueued' = \text{FALSE} \land outside' = \text{FALSE}$
$\land \text{UNCHANGED } \langle host, reports, inherited, selected, changes, contexts, warnings, corrections, wakes, work, outsides \rangle$

$StartTurn \triangleq$
$\land active \land \neg inTurn \land inTurn' = \text{TRUE}$
$\land \neg(host = \text{“opencode2”} \land correctionPending)$
$\land correctionTurn' = \text{FALSE}$
$\land \text{UNCHANGED } \langle host, active, busy, reports, baseline, inherited, selected, request, reply, requests, changes, session, visible, contextPending, contextSent, correctionQueued, contexts, warnings, corrections, wakes, work, correctionPending, outside, outsides \rangle$

$StartCorrectionTurn \triangleq$
$\land active \land \neg inTurn \land correctionPending$
$\land inTurn' = \text{TRUE} \land correctionTurn' = \text{TRUE} \land correctionPending' = \text{FALSE}$
$\land \text{UNCHANGED } \langle host, active, busy, reports, baseline, inherited, selected, request, reply, requests, changes, session, visible, contextPending, contextSent, correctionQueued, contexts, warnings, corrections, wakes, work, outside, outsides \rangle$

$WorkCompleted \triangleq$
$\land active \land work < (MaxSessions + 1) * MaxRequests$
$\land work' = work + \text{IF } correctionTurn \text{ THEN } 0 \text{ ELSE } 1$
$\land correctionQueued' = (correctionTurn \land correctionQueued)$
$\land \text{UNCHANGED } \langle host, active, busy, reports, baseline, inherited, selected, request, reply, requests, changes, session, visible, contextPending, contextSent, contexts, warnings, corrections, inTurn, wakes, correctionPending, correctionTurn, outside, outsides \rangle$

Section: Allowed executions and fairness

Spec composes check-completion fairness with fair response and report servicing. Inspection independence is a safety property: its response remains enabled even while a check is held.

$CheckCompletes \triangleq \exists f \in Snapshots : FinishCheck(f) \lor OutsideFinish(f)$
$Next \triangleq (\exists k \in Kinds : Request(k)) \lor Respond \lor Acknowledge \lor BeginCheck$
$\lor CheckCompletes \lor Report \lor Deliver \lor Stop \lor Restart$
$\lor StartTurn \lor StartCorrectionTurn \lor WorkCompleted$
$\lor (\neg active \land session = MaxSessions \land \text{UNCHANGED } vars)$

Check completion fairness is the *RunsEnd* guarantee of the result model. Inspection is independently enabled; it never needs that guarantee to reply.

$Spec \triangleq Init \land \Box[Next]_{vars} \land \text{WF}_{vars}(CheckCompletes)$
$\land \text{WF}_{vars}(Respond) \land \text{WF}_{vars}(Report) \land \text{WF}_{vars}(Deliver)$

Section: Safety and progress properties

Capabilities constrain completion, editors never acquire agent gates, closed sessions stay silent, pending context stays current, and corrections are deduplicated. Progress properties require the fairness stated in *Spec*.

$InspectionIndependentOfChecks \triangleq$
$active \land request \in \{\text{“inspect”}, \text{“repair”}\} \land \neg reply.ready \Rightarrow \text{ENABLED } Respond$
$InspectionAlwaysAllowed \triangleq$
$request \in \{\text{“inspect”}, \text{“repair”}\} \land reply.ready \Rightarrow reply.decision = \text{“allow”}$

```
EditorNeverGuards ≜
    host = "neovim" ∧ request ≠ "none" ∧ reply.ready ⇒ reply.decision = "allow"
StoppedSessionsStaySilent ≜ ¬active ⇒ ¬contextPending ∧ ¬visible.known
HostsWithoutIdleWakeStayIdle ≜ ¬Capabilities(host).idleWake ⇒ wakes = 0
CorrectionRequestsAreDeduplicated ≜ corrections ≤ changes + session + work + 1
PendingContextIsCurrent ≜ contextPending ⇒ Notify
CompletionUsesHostCapability ≜
    request = "complete" ∧ reply.ready ⇒
        reply.decision ∈ {"allow", Capabilities(host).completion}
EditorOnlyReports ≜
    host = "neovim" ⇒ contexts = 0 ∧ warnings = 0 ∧ corrections = 0 ∧ wakes = 0
PendingReportsEventuallyResolve ≜ contextPending ⇝ ¬contextPending
FinishedReportsBecomeVisible ≜
    (active ∧ ¬busy) ⇝ (visible.known ∨ ¬active)
InspectionEventuallyReplies ≜
    (active ∧ request ∈ {"inspect", "repair"} ∧ ¬reply.ready)
      ⇝ (reply.ready ∨ ¬active)
```